

PW EARNERS
DSA in C++

DPP — 9 (SOLUTIONS)

Topic: Strings

Multiple Choice Questions (Predict the Output & Theory)

Q1. Sorting a String

Correct Option: **B (abc)**

Explanation: The inbuilt `sort()` function rearranges the characters of the string in ascending (lexicographical) order. So, "cba" becomes "abc".

.....

Q2. Time Complexity of `to_string()`

Correct Option: **B ($O(\text{len})$)**

Explanation: The `to_string()` function processes each digit of the number to convert it into a character. Hence, it takes time proportional to the number of digits (`len`).

.....

Q3. Extracting Substring

Correct Option: **A (is)**

Explanation: `substr(6, 2)` means starting from index 6, extract 2 characters. Index 6 is 'i', and the next character is 's', resulting in "is".

.....

Q4. String Mutability

Correct Option: **A (Apple)**

Explanation: Strings in C++ are mutable (can be changed). `s[0] = 'A'` directly updates the first character from lowercase 'a' to uppercase 'A'.

.....

Q5. Most Frequent Character TC

Correct Option: **C ($O(N)$)**

Explanation: By using a frequency array of size 26 or 128, we can simply traverse the string once (taking $O(N)$ time) to count occurrences, making it highly optimized.

.....

Q6. String to Long Long

Correct Option: **B (`stoll()`)**

Explanation: `stoi()` converts to `int`, while `stoll()` is specifically used to convert a string to a `long long` integer.

.....

Q7. Length of Number

Correct Option: **D (5)**

Explanation: `to_string(98765)` converts the number to the string "98765", which has ex-

actly 5 characters.

.....

Q8. Longest Continuous Word Technique

Correct Option: **C (Variable-size Sliding Window / 2-Pointer)**

Explanation: The 2-pointer technique where we jump to the start of the next word dynamically is the standard, most optimal way to extract or measure words in a sentence without using extra space.

.....

Q9. Defanging an IP Address

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 string defangIPAddr(string address) {
6     string ans = "";
7     for (int i = 0; i < address.length(); i++) {
8         if (address[i] == '.') {
9             ans += "[.]";
10        } else {
11            ans += address[i];
12        }
13    }
14    return ans;
15 }
16
17 int main() {
18     string address;
19     cin >> address;
20     cout << defangIPAddr(address) << endl;
21 }
```

Explanation : We create an empty string `ans`. We iterate through the given IP address, and whenever we encounter a '.', we append "[.]" to our answer string. Otherwise, we append the current character.

.....

Q10. Remove All Adjacent Duplicates In String

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 string removeDuplicates(string s) {
6     string ans = "";
7     for (int i = 0; i < s.length(); i++) {
8         // If ans has characters and the last character matches the
9         // current one
10        if (ans.length() > 0 && ans.back() == s[i]) {
11            ans.pop_back(); // Remove the duplicate
12        } else {
13            ans += s[i];
14        }
15    }
16    return ans;
17 }
```

```

12         ans.push_back(s[i]); // Otherwise, add it
13     }
14 }
15 return ans;
16 }
17
18 int main() {
19     string s;
20     cin >> s;
21     cout << removeDuplicates(s) << endl;
22 }

```

Explanation : This problem becomes extremely easy if we treat our answer string like a stack. As we iterate through the original string, we check the last inserted character in our answer string using `ans.back()`. If it matches the current character `s[i]`, it means we found an adjacent duplicate! Instead of adding the current character, we use `ans.pop_back()` to remove the previous one. If they don't match, we push the current character.

Q11. Longest Common Prefix

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5  using namespace std;
6
7  string longestCommonPrefix(vector<string>& strs) {
8      if (strs.size() == 0) return "";
9
10     // Sort the array of strings lexicographically
11     sort(strs.begin(), strs.end());
12
13     string first = strs[0];
14     string last = strs[strs.size() - 1];
15     string ans = "";
16
17     // Compare only the first and last strings
18     for (int i = 0; i < min(first.length(), last.length()); i++) {
19         if (first[i] != last[i]) {
20             break;
21         }
22         ans += first[i];
23     }
24     return ans;
25 }
26
27 int main() {
28     int n;
29     cin >> n;
30     vector<string> strs(n);
31     for (int i = 0; i < n; i++) {
32         cin >> strs[i];
33     }

```

```

34     cout << longestCommonPrefix(strs) << endl;
35 }

```

Explanation : Instead of comparing every single character of every string (which would be very slow and complex), we use a brilliant trick: sorting! When we sort an array of strings, they are arranged in dictionary order. This guarantees that the most differing strings are placed at the very beginning and the very end of the array. Thus, the longest common prefix of the entire array is simply the longest common prefix of the **first** and **last** string.

Q12. Check String Rotation

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  bool rotateString(string s, string goal) {
6      // If lengths do not match, they can never be rotations
7      if (s.length() != goal.length()) {
8          return false;
9      }
10
11     // Concatenate s with itself
12     string doubledString = s + s;
13
14     // Check if goal is a substring of the doubled string
15     if (doubledString.find(goal) != string::npos) {
16         return true;
17     }
18     return false;
19 }
20
21 int main() {
22     string s, goal;
23     cin >> s >> goal;
24     if (rotateString(s, goal)) {
25         cout << "true" << endl;
26     } else {
27         cout << "false" << endl;
28     }
29 }

```

Explanation : Since the constraints are high (10^5), manually shifting characters one by one using nested loops will result in a Time Limit Exceeded (TLE) error because it takes $O(N^2)$ time. The optimal $O(N)$ trick is to concatenate string s with itself ($s + s$). Why? Because $s + s$ inherently contains every single possible rotation of s as a substring! For example, if $s = "abcde"$, $s + s = "abcdeabcde"$. Notice how all valid rotations like "bcdea" and "cdeab" naturally exist inside this new string. We simply use the inbuilt `.find()` function to see if `goal` is inside it.

Q13. Absolute Frequency Difference of Characters

```

1  #include <iostream>

```

```

2  #include <string>
3  #include <vector>
4  #include <cmath>
5  using namespace std;
6
7  string freqDifference(string s1, string s2) {
8      vector<int> freq1(26, 0);
9      vector<int> freq2(26, 0);
10
11      // Populate frequency arrays
12      for (int i = 0; i < s1.length(); i++) freq1[s1[i] - 'a']++;
13      for (int i = 0; i < s2.length(); i++) freq2[s2[i] - 'a']++;
14
15      string ans = "";
16
17      // Calculate difference and construct the result string
18      for (int i = 0; i < 26; i++) {
19          int diff = abs(freq1[i] - freq2[i]);
20          while (diff > 0) {
21              ans += (char)(i + 'a');
22              diff--;
23          }
24      }
25      return ans;
26 }
27
28 int main() {
29     string s1, s2;
30     cin >> s1 >> s2;
31     cout << freqDifference(s1, s2) << endl;
32 }

```

Explanation : We maintain two frequency arrays of size 26 (one for each string) to record how many times each character appears. We then loop from 0 to 25, calculate the absolute difference between counts using `abs()`, and append the respective character to our answer string that many times. Since we loop from 0 to 25 ('a' to 'z'), the output is naturally sorted!

Q14. Sort Characters By Frequency

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  string frequencySort(string s) {
7      vector<int> freq(128, 0);
8      for (int i = 0; i < s.length(); i++) {
9          freq[s[i]]++;
10     }
11
12     string ans = "";
13
14     // Keep finding the maximum frequency until all are 0

```

```

15     while (true) {
16         int maxFreq = 0;
17         int maxIdx = -1;
18
19         for (int i = 0; i < 128; i++) {
20             if (freq[i] > maxFreq) {
21                 maxFreq = freq[i];
22                 maxIdx = i;
23             }
24         }
25
26         // If max frequency is 0, we have placed all characters
27         if (maxFreq == 0) break;
28
29         // Add the character maxFreq times to answer
30         for (int i = 0; i < maxFreq; i++) {
31             ans += (char)maxIdx;
32         }
33
34         // Reset the frequency of this character so we don't pick it
35         // again
36         freq[maxIdx] = 0;
37     }
38
39     return ans;
40 }
41
42 int main() {
43     string s;
44     cin >> s;
45     cout << frequencySort(s) << endl;
46 }

```

Explanation : Without using advanced concepts like hashmaps or custom sorting comparators, we can solve this smartly. We map the frequencies in an array of size 128 (to accommodate all ASCII characters). Then, we run a loop to find the character with the highest frequency, append it to our answer string the required number of times, and manually set its frequency to 0 in the array so we don't pick it again. We repeat this process until the maximum frequency found is 0.

Q15. Group Anagrams

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5  using namespace std;
6
7  void printGroupedAnagrams(vector<string>& strs) {
8      int n = strs.size();
9      vector<bool> visited(n, false);
10     vector<vector<string>> ans;
11

```

```
12     for (int i = 0; i < n; i++) {
13         if (visited[i]) continue;
14
15         vector<string> currentGroup;
16         string s1 = strs[i];
17         sort(s1.begin(), s1.end());
18
19         for (int j = i; j < n; j++) {
20             if (!visited[j]) {
21                 string s2 = strs[j];
22                 sort(s2.begin(), s2.end());
23
24                 // If sorted versions match, they are anagrams
25                 if (s1 == s2) {
26                     currentGroup.push_back(strs[j]);
27                     visited[j] = true;
28                 }
29             }
30         }
31         ans.push_back(currentGroup);
32     }
33
34     for (int i = 0; i < ans.size(); i++) {
35         cout << "[";
36         for (int j = 0; j < ans[i].size(); j++) {
37             cout << ans[i][j];
38             if (j != ans[i].size() - 1) cout << ", ";
39         }
40         cout << "]" << " ";
41     }
42     cout << endl;
43 }
44
45 int main() {
46     int n;
47     cin >> n;
48     vector<string> strs(n);
49     for (int i = 0; i < n; i++) {
50         cin >> strs[i];
51     }
52     printGroupedAnagrams(strs);
53 }
```

Explanation : Because the constraints are small ($N \leq 100$), we do not need to use HashMaps. We can create a boolean `visited` array to track which strings have already been grouped. We iterate over the array, and if a string hasn't been grouped, we sort it. Then, we check all upcoming unvisited strings by sorting them as well. If the sorted forms match, it guarantees they are anagrams (they share the exact same characters). We push them into our current group and mark them as visited!